Unit 6: Exception Handling, Java IO and Streams 7 hrs.

6.1 Managing Errors and Exceptions
6.2 Syntax of Exception Handling Code
6.3 Types of Exceptions
6.4 Using try and catch keywords
6.5 Multiple catches
6.6 Using Finally Statement
6.7 Throwing Exceptions
6.8 Streams: Byte Streams and Character Streams
6.9 Reading / Writing Console Input / Output
6.10 Reading and Writing files

Exception handling

Error

It is common to make mistakes while developing as well as typing a program. A mistake might lead to an error causing the program to produce unexpected results. An error may produce an incorrect output or may terminate the execution of the program abruptly of even may cause the system to crash. It is therefore important to detect and manage properly all the possible error conditions in the program so that the program will not terminate or crash during execution.

Types of errors

Errors are broadly classified into two categories:

- Compile time errors
- Run-time errors

Compile time errors

Those errors that encounter at compilation time are known as compile time errors. All syntax errors will be detected and displayed by the java compiler and therefore these errors are known as compile

time errors. Whenever the compiler displays an error, it will not create the .class file. It is therefore necessary that all the errors should be fixed before successfully compile and run the program. Most of the compile time errors are due to typing mistakes. The most common compile time errors are:

- Missing semicolon
- Missing (or mismatch of) brackets in classes and methods
- Misspelling of identifiers and keywords
- Missing double quotes in string
- Use of undeclared variables
- Incompatible types in assignment/initialization
- Bad reference to objects
- And so on.

Run time errors

All the errors that encounter at run time are known as run time errors. During runtime errors a program may compile successfully creating the .class file but may not run properly. Such

programs may produce wrong results due to wrong logic or may terminate due to errors. Most common runtime errors are:

- Dividing an integer by zero.
- Accessing an element that is out of the bounds of an array.
- Trying to store a value into an array of an incompatible class or type.
- Trying to cast an instance of a class to one of its subclasses
- Passing a parameter that is not in a valid range or value for a method
- Trying to illegally change the state of a thread
- Attempting to use a negative size for an array
- Converting invalid string to a number
- Accessing a character that is out of bounds of a string
- And so on.

Exceptions

An exception is a condition that is caused by a runtime error in the program. When the Java interpreter encounters an error such as dividing an integer by zero, it creates an exception object and throws (i.e., inform us that an error has occurred).

If the exception object is not caught and handled properly, the interpreter will display an error message and will terminate the program. To continue the program execution, we should try to catch the exception object thrown by the error condition and then display an appropriate message for taking corrective actions. This task is known as exception handling. The purpose of exception handling is to provide a means to detect and report an exceptional circumstance so that appropriate

action can be taken.

Java exception handling is managed by five keywords: **try, catch, throw, throws and finally**

Program statements that may generate exception are contained within **try** block. If an exception occurs within the try block, it is thrown. Those statements that are used to catch those exceptions are kept in **catch** block and handled in some relational manner. System generated exceptions are automatically thrown by the java runtime system. To manually throw an exception, use the keyword **throw**. Any exception that is thrown out of a method must be specified as such by a **throws** clause. Any code that absolutely must be executed before a method returns is put in a **finally** block.

The general syntax of exception handling block is:

```
try{

  // block of code to monitor for errors

}catch(Exception_type1 exob){

  //exception handler for exception type1

}catch(Exception_type2 exob){

  //exception handler for exception type2
}
finally{

  //block of code to be executed before try block ends
}
```

Here, exception type is the type of exception that has occurred.

Exception type

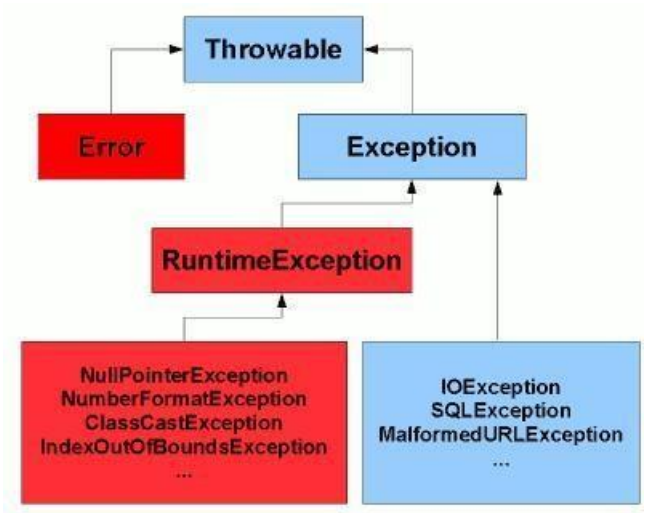

Fig: exception hierarchy

All exception types are subclasses of the built-in class **Throwable.** Thus Throwable is at the top of the exception class hierarchy. Immediately below Throwable are two subclasses that partition exceptions into two distinct branches: error and exception. There is an important subclass of exception, called **RuntimeException**. The other branch called error, which define exceptions that are not expected to be caught under normal circumstances by the program.

Using try and catch

The code that may generate exception is enclosed within try block. Immediately following try block, include a catch clause that specifies the exception type that you wish to catch.

Program which includes try and catch block that processes the ArithmeticException generated by the division-by-zero error.

```
class Exception {
    public static void main(String args[]) {
        int a, b;
        try {
            b = 0;
            a = 12 / b;
            System.out.println(" Inside try block");
        // this will not be printed
        } catch (ArithmeticException e) {
        //catch division by zero error
            System.out.println("Division by zero.");
        }
        System.out.println("Outside try and catch block");
    }
}
```

Output

Division by zero.

Outside try and catch block

Multiple catch clauses

In some cases, more than one exception could be raised by a single piece of code. To handle this

type of situation, we can specify two or more **catch** clauses, each catching a different type of exception. When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed. After one **catch** statement executes,

the others are bypassed and execution continues after the **try/catch** block.

Syntax;

try{

Statements;

}

catch (exception_type1 e){

Statements;

}**catch** (exception_type2 e){

Statements;

}

……

……..

catch (exception_typeN e){

Statement;

}

Write a program that demonstrate the concept of multiple catch blocks

```
class MultipleCatch {
    public static void main(String args[]) {
        int a[] = {5, 10};
        int b = 5;
        try {
            int x = a[2] / (b - a[1]);
            int y = a[1] / a[0];
            System.out.println("Y= " + y);

        } catch (ArithmeticException e) {
            System.out.println("Division by zero");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index error");
        } catch (ArrayStoreException e) {
            System.out.println("Wrong data type");
        }
    }

}
```

Output

Array index error Y=2

finally statement

Java supports another statement known as **finally** statement that can be used to handle an exception that is not caught by any of the previous **catch** statements. **finally** block can be used to handle any exception generated within a **try** block. It may be added immediately after the **try** block or after the last **catch** block.

```
try{


}
finally{


}

Or
try{

          .........
          .........


          ..........
          ..........




          ..........

          ..........
}catch(.........){
          ............
          ............
}
catch(){



}finally {

          ..........
          ..........
          ............
```

```
            ............
}
```

throw

When we would like to throw our own exceptions, we can do this by using the keyword throw. The general syntax of throw is as:

throw new throwableinstance;

Here, throwableinstance must be an object of type Throwable or a subclass of throwable. The flow of execution stops immediately after the throw statement, any subsequent statements are not executed.

Program that demonstrate the concept of throw (Custom Exception)

```
class MyException extends Exception {
    MyException(String message) {
        super(message);
    }
}

class TestMyException {
    public static void main(String args[]) {
        int x = 5, y = 1000;
        try {
            float z = (float) x / (float) y;
            if (z < 0.01) {
                throw new MyException("Number is too small");
            }
        } catch (MyException e) {
            System.out.println("Caught  my Exception");
            System.out.println(e.getMessage());
        } finally {
            System.out.println("I am always here");
        }
    }
}
```

Output

Caught my Exception Number is too small I am always here

<u>throws clause</u>

The throws keyword appears at the end of a method's signature. A **throws** clause lists the types

of exceptions that a method might throw.
The general form of a method declaration that includes a **throws** clause:

```
type methodName(parameter-list) throws exception-list{
// body of method
}
```

Example showing the throws clause:

```
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        System.out.println("Inside throwOne.");
        throw new IllegalAccessException("demo");
    }

    public static void main(String args[]) {
        try {
            throwOne();
        } catch (IllegalAccessException e) {
            System.out.println("Caught " + e);
        }
    }
}
```

Output:
Inside throwOne
caught java.lang.IllegalAccessException: demo

Streams:

A stream is a sequence of data. IO stream refers to a stream that is unlikely a method to sequentially access a file. I/O Stream means an input source or output destination representing different types of sources e.g. disk files. The java.io package provides classes that allow you to convert between Unicode character streams and byte streams of non-Unicode text.

- Input Stream: reads data from the source.
- Output Stream: writes data to a destination.

All the input and output operations are handled by **java.io** package. All the classes that are needed to perform input/output operations are contained inside java.io package. A stream can be defined as a sequence of data. A stream produces or consumes information. A stream is linked to a physical device by the Java I/O system. The InputStream is used to read data from a source and the OutputStream is used for writing data to a destination.

Byte/Character Stream:

Byte Stream is used to perform input/output of 8-bit bytes. Following example explains the logic of byte stream where both the read and write operations are carried out in the same program.

```
import java.io.*;
public class CopyFile {
    public static void main(String[] args) {
        try {
            FileInputStream in =
                   new FileInputStream("input.txt");
            FileOutputStream out =
                   new FileOutputStream("output.txt");
            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }
            in.close();
            out.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

In above program we open the file input.txt for reading the content by creating the object of the FileInputStream class and the file output.txt for writing the content by creating the object of the FileOutputStream class. After that we initialized the variable „c" and read the content from the file input.txt by using read() method and then we write the content by invoking the write() method to the file output.txt.

Character Stream is used to perform input/output of 16-bit Unicode. Most frequently used classes in character stream are **FileReader** and **FileWriter.** Above example can be written using FileReader and FileWriter as follows;

```
import java.io.*;
public class CopyFile {
    public static void main(String args[]) {
        try {
            FileReader in = new FileReader("input.txt");
            FileWriter out = new FileWriter("output.txt");
            int c;
            while ((c = in.read()) != -1) {
```

```
                out.write(c);
            }
            in.close();
            out.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Another example of FileReader FileWriter class is as shown below;

```
import java.io.*;
public class FileRead {
    public static void main(String args[]) {
        try {
            File file = new File("Hello1.txt");
            file.createNewFile();
            FileWriter writer = new FileWriter(file);
            writer.write("This \n is \n an \n example\n");
            writer.flush();
            writer.close();

            FileReader fr = new FileReader(file);
            char[] a = new char[50];
            fr.read(a);
            for (char c : a) {
                System.out.println(c);
            }
            fr.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```